

# Università degli Studi di Salerno

Corso di Ingegneria del Software

## Tasky Object Design Document Versione 0.5



Data: 19/12/2025



|                                   |                  |
|-----------------------------------|------------------|
| Progetto: Tasky                   | Versione: 0.5    |
| Documento: Object Design Document | Data: 19/12/2025 |

**Coordinatore del progetto:**

| Nome                           | Matricola  |
|--------------------------------|------------|
| Luca Vincenzo Lambiase Fontana | 0512113338 |
|                                |            |

**Partecipanti:**

| Nome          | Matricola  |
|---------------|------------|
| Ilaria Lastra | 0512120280 |
| Rinaldo Perna | 0512119593 |
|               |            |
|               |            |
|               |            |
|               |            |

|                    |               |
|--------------------|---------------|
| <b>Scritto da:</b> | Ilaria Lastra |
|--------------------|---------------|

**Revision History**

| Data       | Versione | Descrizione                                                                                                                                                                                | Autore                                                       |
|------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 01/10/2025 | 0.1      | Presentazione Proposta di Progetto                                                                                                                                                         | Lastra Ilaria                                                |
| 10/10/2025 | 0.2      | Presentazione Problem Statement con descrizione degli scenari principali dell'applicazione, i suoi requisiti e gli ambienti target                                                         | Lastra Ilaria                                                |
| 20/10/2025 | 0.3      | Presentazione del Requirements Analysis Document, con perfezionamento dei requisiti esposti nel Problem Statement e definizione dei criteri di successo e dello scopo e ambito del sistema | Lastra Ilaria                                                |
| 08/11/2025 | 0.3.1    | Presentazione dell'Object Model e del Dynamic Model, con Class Diagram, Sequence Diagram e Statechart Diagram relativi al Requirements Analysis Document                                   | Lastra Ilaria                                                |
| 19/11/2025 | 0.4      | Presentazione del System Design Document, con discussione dei design goals, dell'architettura e mappatura hardware, software.                                                              | Lastra Ilaria, Luca Vincenzo Lambiase Fontana, Rinaldo Perna |
| 19/12/2025 | 0.5      | Presentazione dell'Object Design Document con discussione dei compromessi della progettazione ad oggetti, linee guida per la documentazione dell'interfaccia e packaging dei sottosistemi  | Ilaria Lastra, Luca Vincenzo Lambiase Fontana, Rinaldo Perna |
|            |          |                                                                                                                                                                                            |                                                              |



## Sommario

|      |                                               |   |
|------|-----------------------------------------------|---|
| 1.   | Introduzione.....                             | 6 |
| 1.1. | Object design trade-offs .....                | 6 |
| 1.2. | Interface documentation guidelines .....      | 6 |
| 1.3. | Definitions, acronyms and abbreviations ..... | 7 |
| 1.4. | References .....                              | 7 |
| 2.   | Packages .....                                | 7 |
| 3.   | Class Interface Glossary .....                | 9 |

## 1. Introduzione

Il seguente documento è volto a presentare L'Object Design, il quale rappresenta la specifica dettagliata dell'architettura software a livello degli oggetti, definendo come i sottosistemi parte della nostra applicazione vengono realizzati concretamente attraverso classi, interfacce e interazioni. Qui rappresentiamo una guida definitiva per la fase di codifica (Implementation), garantendo che le decisioni architettoniche siano tradotte in codice in modo coerente e manutenibile.

### 1.1. Object design trade-offs

I principali trade-off, ossia decisioni strategiche prese per risolvere i conflitti tra requisiti non funzionali contrastanti per la nostra applicazione Tasky sono:

- **Buy VS Build:** Abbiamo deciso di utilizzare componenti “off-the-shelf” consolidati per la gestione dei dati (MySQL) e per il framework web (Flask) concentrando lo sviluppo, ossia la Build, sulla logica di business specifica di Tasky (gestione dei progetti, task e dipendenti).
- **Gestione della Sessione (Stateless VS Stateful):** A differenza delle webapp tradizionali, TaskyAPI adotta un approccio Stateless. L'autenticazione avviene tramite Token (inviati nell'header o nel body delle richieste). Questo elimina la necessità di mantenere sessioni in memoria sul server (HttpSessioni), rendendo l'API scalabile e facilmente consumabile da client diversi (App Android, Web Client).
- **Database Mapping:** Si è scelto di non utilizzare un ORM complesso (come SQLAlchemy ORM) ma di gestire le query SQL direttamente tramite helper functions (repository.py) e driver PyMySQL. Questo garantisce il controllo totale sulle performance delle query e riduce l'overhead, mappando i risultati direttamente in dizionari Python o DTO (Marshmallow Schemas).

### 1.2. Interface documentation guidelines

La coerenza del codice è l'elemento che riduce principalmente la probabilità di errori durante l'integrazione. Per la realizzazione dell'implementazione della nostra applicazione, sono rigorosamente richieste delle specifiche convenzioni.

Per quanto riguarda gli standard di naming:

- **Classi:** I nomi delle classi vengono indicati usando sostantivi singolari e utilizzando nomenclatura PascalCase (es. DipendenteDAO)
- **Metodi:** Per i metodi utilizziamo una notazione camelCase (es. aggiungiTask)
- **Variabili:** Anche per le variabili utilizziamo una notazione camelCase
- **Costanti:** per le costanti utilizziamo UPPER\_SNAKE\_CASE (es. MAX\_VALUE)
- **Package:** i nomi dei pacchetti dovranno essere in minuscolo e separati da punti (es. tasky.pacchetto)

Per la gestione degli errori e la robustezza del sistema:

- Gli errori non vengono gestiti tramite codici di ritorno, ma tramite il lancio di EccezioniCustom(es. ValidationException) che vengono catturate da handler centralizzati per restituire risposte JSON coerenti.
- L'input viene validato rigorosamente tramite Marshmallow Schemas prima di raggiungere la logica di business.

Infine, per la gestione delle collezioni all'interno del sistema:

### ***1.3. Definitions, acronyms and abbreviations***

| TERMINE    | DEFINIZIONE                                                                                                                                                                                                                                                               |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DAO</b> | Il <b>DAO (Data Access Object)</b> è un design pattern strutturale il cui scopo è isolare la logica di business (o logica applicativa) dallo strato di persistenza dei dati (il database, file XML, o web service).                                                       |
| <b>ORM</b> | <b>(Object-Relational Mapping)</b> è una tecnica di programmazione (e spesso un framework software) che permette di far interagire un linguaggio di programmazione orientato agli oggetti (come Java, C# o Python) con un database relazionale (come MySQL o PostgreSQL). |
| <b>DTO</b> | (Data Transfer Object) è un design pattern il cui scopo principale è il raggruppamento di dati per ottimizzare il trasferimento tra diverse parti di un sistema, specialmente quando queste comunicano attraverso una rete.                                               |
| <b>PK</b>  | (Primary Key) Chiave Primaria, ossia un campo o un insieme di campi che identifica in modo univoco e inequivocabile ogni riga all'interno di una tabella                                                                                                                  |
| <b>FK</b>  | (Foreign Key) Chiave Esterna, ossia lo strumento che permette di creare un legame logico tra due tabelle diverse.                                                                                                                                                         |

### ***1.4. References***

- Università degli Studi di Salerno - Corso di Ingegneria del Software - “Proposta di Progetto Tasky”
- Università degli Studi di Salerno - Corso di Ingegneria del Software - “Problem Statement”
- Università degli Studi di Salerno - Corso di Ingegneria del Software - “Requirements Analysis Document Tasky”
- Università degli Studi di Salerno - Corso di Ingegneria del Software - “System Design Document”

## **2. Packages**

Il sistema Tasky è decomposto nei seguenti package, con lo scopo di massimizzare la coesione e minimizzare l'accoppiamento tra le componenti:

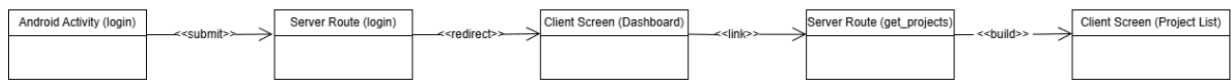
| Modulo/File                   | Ruolo nel Design         | Responsabilità principale                                                                                                              |
|-------------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>      | Application Factory      | Inizializza l'app Flask, registra i Blueprint e configura lo scheduler                                                                 |
| <code>routes.py</code>        | Controller / Boundary    | Espone gli endpoint REST (bd.route). Riceve JSON, chiama gli Schema per validare, delega al Manager/Repository e restituisce JSON HTTP |
| <code>manager.py</code>       | Service / Control        | (Livello intermedio per logica di business complessa che non riguarda solo il DB.                                                      |
| <code>repository.py</code>    | DAO (Data Access Object) | Esegue query SQL grezze (cursor.execute). Gestisce transazioni e connessioni al DB. Non sa nulla di HTTP.                              |
| <code>schemas.py</code>       | DTO / Validator          | Definisce gli schemi Marshmallow per validare i dati in ingresso (es. TaskCreateSchema) e serializzare quelli in uscita.               |
| <code>db.py</code>            | Infrastructure           | Gestisce la connessione fisica al database (MySQL/SQLite connector).                                                                   |
| <code>utils.py</code>         | Utility / Security       | Contiene decoratori per l'autenticazione (@manager_required, @member_of_department) e funzioni di hashing password.                    |
| <code>notifications.py</code> | External Adapter         | Gestisce l'integrazione con Firebase Cloud Messaging (FCM) per le notifiche push.                                                      |
| <code>scheduler.py</code>     | Background Service       | Configura e gestisce i job periodici (APScheduler) per il controllo scadenze.                                                          |

## 2.1 Struttura dei pacchetti



## 2.2 Flow chart diagram del sistema





### 3. Class Interface Glossary

Qui di seguito vi è la descrizione delle principali classi presenti all'interno del nostro sistema.

| Classi     | Attributi                                                                                                                                                                                                                                                                     | Descrizione                                                                                                                               |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Dipendente | <ul style="list-style-type: none"> <li>email (String PK)</li> <li>password (String, hashed)</li> <li>nome (String)</li> <li>cognome (String)</li> <li>data_nascita (Date)</li> <li>sex (String)</li> <li>numero_telefono (String)</li> <li>id_department (int, FK)</li> </ul> | Rappresenta un utente generico del sistema                                                                                                |
| Manager    | <ul style="list-style-type: none"> <li>anni_lavorativi (Int)</li> <li>(attributi dipendente)</li> </ul>                                                                                                                                                                       | Manager estende dipendente con privilegi amministrativi, aggiungendo logica relativa alla possibilità di creare progetti e assegnare task |
| Progetto   | <ul style="list-style-type: none"> <li>id_progetto (Int, PK)</li> <li>nome (String)</li> <li>descrizione (String)</li> <li>budget (Float)</li> <li>data_inizio (Date)</li> <li>data_fine (Date)</li> <li>id_department (Int, FK)</li> </ul>                                   | Rappresenta un progetto aziendale                                                                                                         |
| Task       | <ul style="list-style-type: none"> <li>id (Int, PK)</li> <li>nome (String)</li> <li>descrizione (String)</li> <li>stato (String: 'Nuovo', 'In Corso', 'Completato')</li> </ul>                                                                                                | Unità di lavoro assegnata a un dipendente                                                                                                 |

|  |                                                                                                                                                                              |  |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
|  | <ul style="list-style-type: none"> <li>• data_inizio (Date)</li> <li>• data_fine (Date)</li> <li>• id_progetto (Int, FK)</li> <li>• email_dipendente (String, FK)</li> </ul> |  |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

#### 4. Glossary

| TERMINE                          | DEFINIZIONE                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tasky</b>                     | Applicazione Task Manager aziendale                                                                                                                                                                                                                                                                                                            |
| <b>API RESTful</b>               | Un'interfaccia di programmazione applicativa (API) che aderisce ai principi di progettazione dell'architettura <b>REST (Representational State Transfer)</b> .                                                                                                                                                                                 |
| <b>Tasky API RESTful</b>         | Interfaccia RESTful esposta dal nostro server di backend realizzata tramite il framework Flask                                                                                                                                                                                                                                                 |
| <b>Three – tier architecture</b> | L'architettura <b>Three-Tier</b> è l'evoluzione del modello Client-Server ed è lo standard per le applicazioni enterprise e web moderne. Suddivide l'applicazione in tre strati logici (e spesso fisici) distinti: Presentation Tier (Livello di Presentazione), Application Tier (Livello Logico o Business Logic) e Data Tier (Livello Dati) |
| <b>Schema Marshmallow</b>        | Classe responsabile della validazione e serializzazione/deserializzazione dei dati (DTO).                                                                                                                                                                                                                                                      |
| <b>Token</b>                     | Stringa crittografata utilizzata per identificare l'utente senza sessione server-side.                                                                                                                                                                                                                                                         |
| <b>Entity</b>                    | Rappresenta l' <b>informazione persistente</b> e a lungo termine gestita dal sistema.                                                                                                                                                                                                                                                          |
| <b>Token</b>                     | Stringa crittografata utilizzata per identificare l'utente senza sessione server-side.                                                                                                                                                                                                                                                         |
| <b>Trade – offs</b>              | Una decisione strategica in cui si accetta di sacrificare o ridurre una determinata qualità o risorsa del sistema per ottenerne un vantaggio in un'altra.                                                                                                                                                                                      |

